

Project – Report

DoorDash ETA Prediction



Group 6 members: -

PXA240008 - Pratik Alkutkar

VKC240000 - Vaishnavi Chaudhari

DSA230007 - Deepali Attavar

RXM230023 - Ramesh Mohan

FXS230031 - Firasuddin Syed

INDEX

Sl. No.	Title	Page Number
1.	Executive Summary	3
2.	Introduction	3
3.	Data	4
4.	Analysis	5
5.	Results	6
6.	Discussion	12
7.	References	13

1. Executive Summary

In our project, we aimed to improve the accuracy of estimated delivery times (ETAs) on the DoorDash platform using various machine learning algorithms. Late deliveries can cause customer dissatisfaction and churn, which directly impacts DoorDash's business. Our objective is to use predictive modelling techniques to build a robust system that forecasts delivery times based on historical and contextual data. We evaluated multiple regression algorithms, including Linear Regression, Ridge Regression, Random Forest, and XGBoost, across three versions of data processing.

Over the course of the project, we applied iterative improvements including missing value handling, data encoding, outlier removal, PCA, feature engineering, and hyper-parameter tuning. Among the models, the XGBoost model demonstrated the best performance, achieving an R^2 of 0.4938 and RMSE of 10.23 after comprehensive tuning and pre-processing.

This project provides a data-driven foundation for optimizing DoorDash's logistics by offering improved delivery time predictions. Our findings not only identify key factors influencing delivery duration but also demonstrate how advanced ML workflows can yield tangible improvements over naive or linear modeling techniques. Ultimately, this approach can help DoorDash reduce operational costs, improve customer experience, and boost platform loyalty.

2. Introduction

Accurate ETA predictions are critical in food delivery services. According to a Voxware study, 43% of customers abandon a platform after just two late deliveries. Given this, DoorDash must ensure timely deliveries to maintain customer satisfaction and operational efficiency. The motivation for this project stems from the real-world business impact of late deliveries and the opportunity to use machine learning models to enhance ETA prediction.

This project focuses on developing a data-driven approach to better estimate delivery duration using machine learning techniques. We also aim to identify the most influential factors affecting delivery time, which can help optimize both logistics and customer experience.

Furthermore, with the growth of gig economy platforms, the ability to forecast delays in real time is increasingly valuable. Companies like DoorDash operate in complex, high-variance environments involving traffic, restaurant prep time, and fluctuating dasher availability. Traditional rule-based systems are often insufficient for capturing this variability. Even though we do not have this data available, through this project, we explore how a scalable machine learning pipeline can be used to dynamically adapt to these variables and offer more reliable ETA estimates that can be acted upon in real-time or used to inform strategic decisions.

3. Data: Data Summary, Description, and Visualization

Data Source: Kaggle

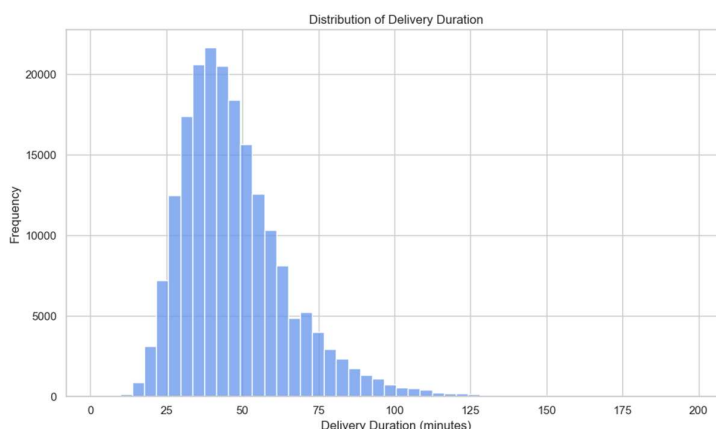
Size: 197,428 rows × 16 columns

Dataset Features:

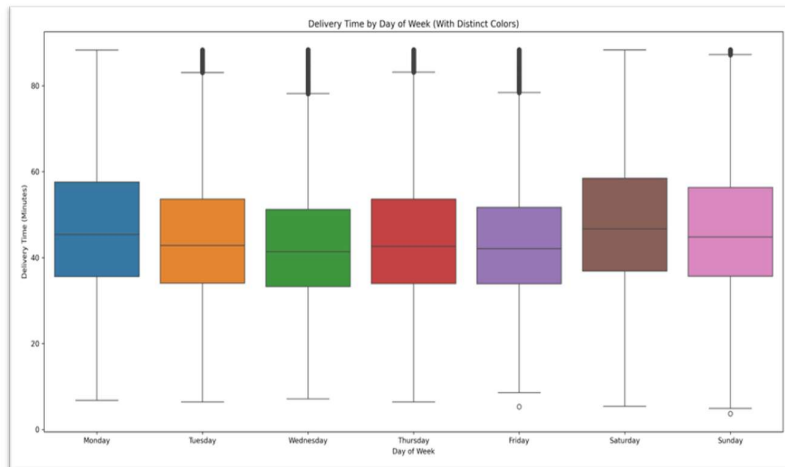
1. **market_id** – Numeric ID representing the city or region of the order.
2. **created_at** – UTC timestamp of when the order was placed.
3. **actual_delivery_time** – UTC timestamp of when the order was delivered.
4. **store_id** – Numeric ID identifying the restaurant.
5. **store_primary_category** – The cuisine type offered by the restaurant
6. **order_protocol** – The ordering channel used (e.g., app, website, API).
7. **total_items** – Total number of items in the order.
8. **subtotal** – Total cost of the order in cents.
9. **num_distinct_items** – Number of unique items ordered.
10. **min_item_price** – Price (in cents) of the cheapest item in the order.
11. **max_item_price** – Price (in cents) of the most expensive item in the order.
12. **total_onshift_dashers** – No. of dashers available in a 10-mile radius at time of order.
13. **total_busy_dashers** – Number of dashers actively delivering at the time of order.
14. **total_outstanding_orders** – No. of active orders in the region at time of placement.
15. **estimated_order_place_duration** – Estimated time(sec) for restaurant to receive order.
16. **estimated_store_to_consumer_driving_duration** – Predicted drive time from the store to the customer.

Target Variable:

delivery_duration_minutes = $(\text{actual_delivery_time} - \text{created_at}).\text{total_seconds}() / 60$

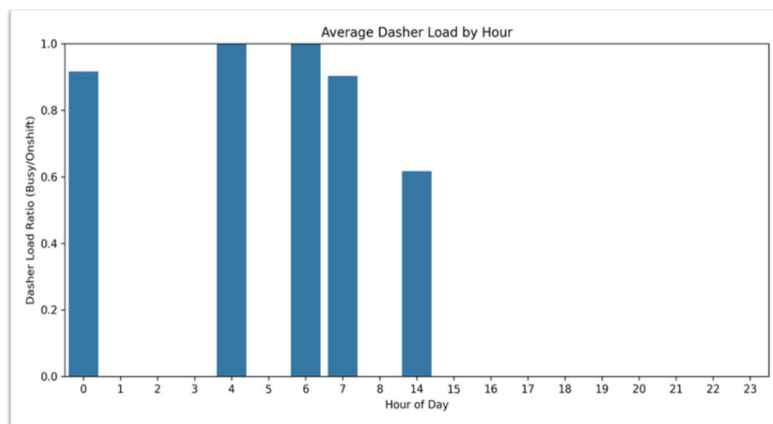


The distribution of delivery duration is right-skewed, with the majority of orders delivered between 30 to 60 minutes. A long tail indicates occasional outliers and significantly delayed deliveries, which may result from high traffic, complex orders, or dasher shortages. This skewness highlights the importance of pre-processing techniques like outlier removal to improve model accuracy.



Delivery Time by Day of Week:

The box plot shows that delivery times are generally longer and more variable on weekends, especially Saturday, while Wednesday and Thursday have the shortest and most consistent delivery duration.



Average Dasher Load by Hour:

The bar chart shows that dasher load peaks during early morning hours (4–6 AM), indicating high delivery demand or limited dasher availability during those times.

4. Analysis

To analyze and predict delivery times, we experimented with the following machine learning models:

- **Linear Regression:** Chosen for its simplicity and interpretability, serving as a baseline model to understand linear relationships between features and delivery time.
- **Ridge Regression:** Selected to improve upon Linear Regression by adding L2 regularization, which helps reduce overfitting in the presence of multicollinearity.
- **Random Forest:** Used for its ability to model complex, non-linear interactions between features through ensemble learning, though it can be sensitive to noise and overfitting.
- **XGBoost:** Implemented for its strong performance in predictive tasks due to gradient boosting, regularization, and handling of missing data efficiently.

We conducted three major analysis iterations:

Version 1: Basic preprocessing with null value removal and encoding. This served as a baseline to understand how the models perform with minimal preprocessing and to identify initial data quality issues.

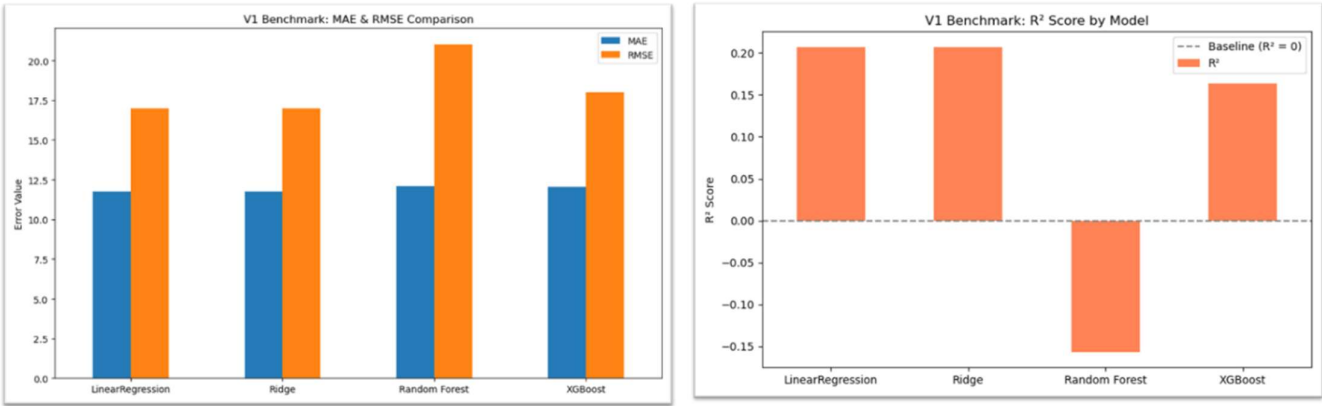
Version 2: Added outlier removal using the IQR method and applied PCA to reduce dimensionality (retained ~95% variance using 6 components). This simplified feature space for better model performance.

Version 3: Enhanced preprocessing with smart missing value imputation and added feature engineering (e.g., average delivery speed, dashers per order). This version addresses limitations in earlier versions by applying more nuanced data cleaning and enrichment techniques. The new engineered features capture order complexity, delivery dynamics, and pricing patterns—factors not fully leveraged in previous versions.

5. Results

a) Results from the benchmark without Pre-Processing

We evaluated 4 models for numeric data only (as no preprocessing is done): Linear Regression, Ridge Regression, Random Forest, and XGBoost on raw data without applying any pre-processing steps.



Model	MAE	RMSE	R2
Linear	11.77	17	0.2072
Ridge	11.77	17	0.2072
Random Forest	12.08	21	-0.1566
XGBoost	12.05	18	0.1632

The results obtained represent how the models performed “out-of-the-box” on the raw data. Overall, models had modest accuracy and high prediction error, indicating a strong need for pre-processing and feature engineering.

b) Pre-processing Steps and Accuracy Change

Version 1: Basic Cleaning (Remove Nulls, Encode Categories)

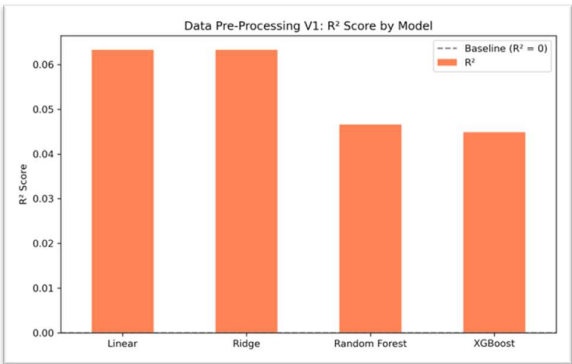
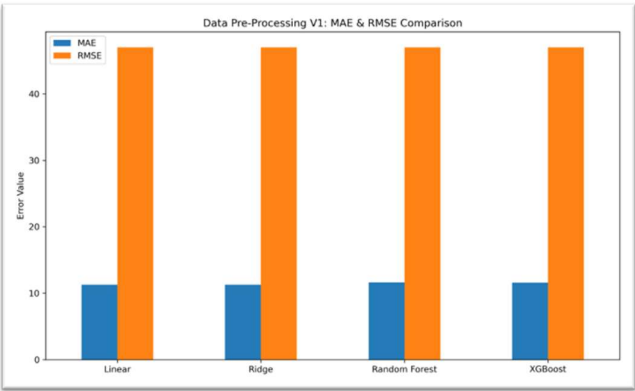
In our first pre-processing iteration, we aimed to clean the data and convert categorical variables into a numerical format suitable for machine learning models.

Pre-processing Steps:

- 1. Missing Value Removal: We removed rows with missing values from important columns that were essential for model training and accurate prediction.
- 2. Target Encoding: We used target encoding on store_primary_category and store_id, this method replaced each category with the average delivery duration observed in the training data, providing a compact numerical representation of categorical information.
- 3. One-Hot Encoding: For categorical variables with fewer unique values such as market_id and order_protocol, we applied one-hot encoding.

Model Performance:

Model	MAE	RMSE	R2
Linear	11.3	47	0.0633
Ridge	11.3	47	0.0633
Random Forest	11.6	47	0.0466
XGBoost	11.6	47	0.0449



Observed Changes in Model Performance:

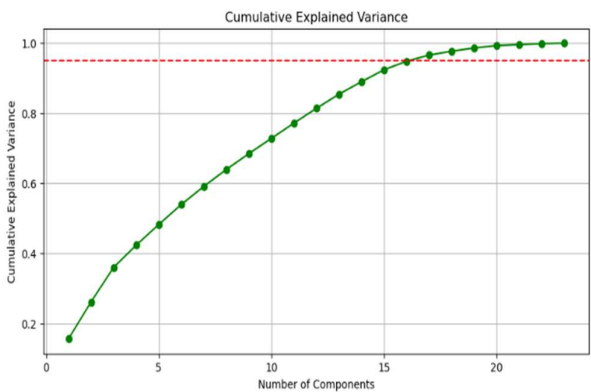
- R² dropped significantly for all models except Random Forest, indicating a loss in explanatory power.
- RMSE spiked to ~47 minutes across all models post-processing, reflecting larger variability in error.
- Version 1 revealed that naive null removal and encoding reduced model performance, highlighting the need for smarter handling. Version 2 addressed this by introducing outlier removal and dimensionality reduction to improve predictive accuracy.

Version 2: Null Removal, Encoding, Outlier Removal, and Dimensionality Reduction

In Version 2, we built upon the foundation of Version 1 by retaining null value removal and encoding but introduced two impactful enhancements: outlier removal and Principal Component Analysis (PCA) for dimensionality reduction.

Pre-processing Steps:

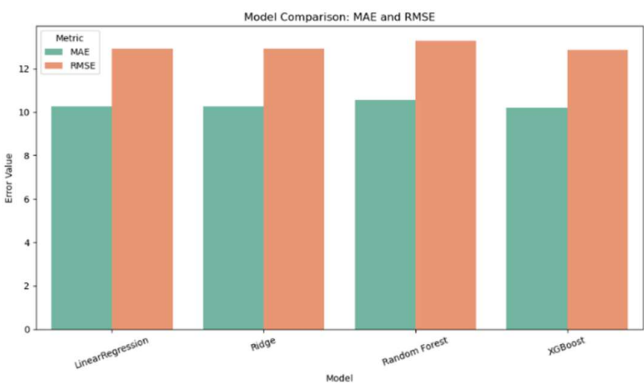
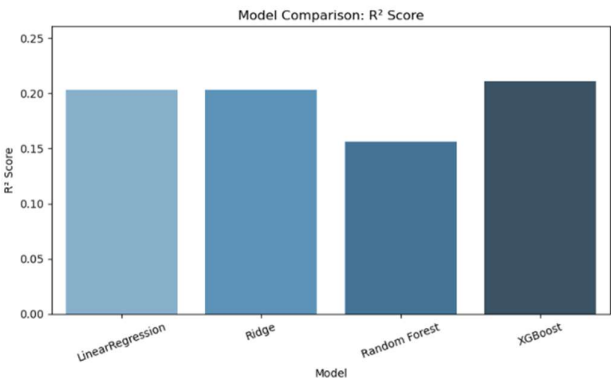
- 1. Outlier Removal (IQR Method):
We applied the Interquartile Range (IQR) method to key numerical features to identify and remove extreme outliers. This step aimed to prevent models from being distorted by a few unusually long deliveries or unrealistic marketplace conditions.
- 2. Dimensionality Reduction via PCA:



After encoding, we had a high-dimensional dataset. PCA was used to reduce this to ~5 principal components, capturing ~95% of the total variance (as seen in the explained variance plot). PCA helped compress correlated features, improving training efficiency and reducing overfitting risk.

Model Performance:

Model	MAE	RMSE	R2
Linear	10.26	12.92	0.2028
Ridge	10.26	12.92	0.2028
Random Forest	10.55	13.29	0.1561
XGBoost	10.21	12.85	0.2105



Observed Changes in Model Performance:

- R^2 scores improved significantly, rising to ~ 0.20 for Linear, Ridge, and XGBoost models, indicating better ability to explain variance in delivery duration.
- RMSE dropped sharply from ~ 47 minutes to ~ 13 minutes, showing a major reduction in large prediction errors after removing extreme outliers.
- Random Forest showed notable gains, improving from negative R^2 to 0.1561 , confirming it could now extract meaningful patterns from cleaner data.

PCA Performance:

Although PCA reduced dimensionality and retained $\sim 95\%$ of variance using 17 components, it did not significantly improve model accuracy ($R^2 = 0.2368$, $RMSE = 12.64$). The components lacked strong correlation with the target and reduced interpretability. As a result, we discontinued PCA after Version 2 and focused on direct feature engineering in Version 3 instead.

Key Takeaway from Version 2:

- Outlier removal significantly improved model performance by reducing extreme errors and enhancing generalization.
- Despite improvements, $R^2 \sim 0.21$ indicated that more predictive signal was needed. Version 3 addressed this by retaining data through intelligent imputation and introducing domain-driven feature engineering to boost model learning.

Version 3: Smarter Imputation and Feature Engineering**Pre-processing Steps:**

In V3, we revisited the handling of missing values and introduced several engineered features to provide additional predictive power.:

1. Smarter Imputation:

- Filled missing `store_primary_category` with "Uncategorized" and `order_protocol` with mode.
- Imputed missing numeric fields like `estimated_store_to_consumer_driving_duration` using median, and `dasher/order` metrics using mean.

2. Advanced Feature Engineering:**a. Avg_delivery_time: Average Delivery Time per Store-Hour**

The average delivery duration for each combination of `store_id` and `hour_of_day`.

This feature captures store-specific performance patterns, some stores consistently take longer to fulfil orders due to kitchen delays, staffing, or location. Including `hour_of_day` adds a temporal layer, accounting for time-based trends like rush hours or late-night delays.

b. `dashers_per_order`: Ratio of dashers to outstanding orders. The number of available dashers divided by the number of pending orders.

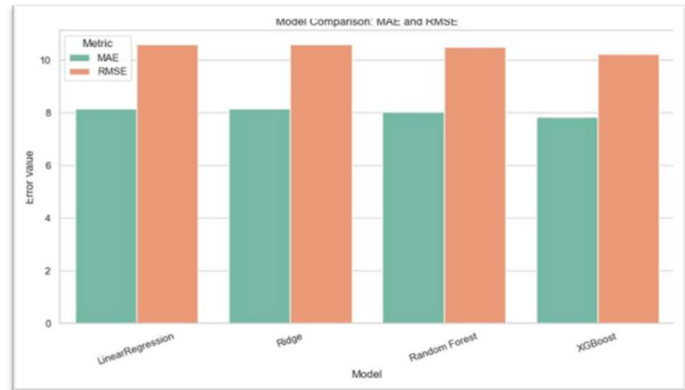
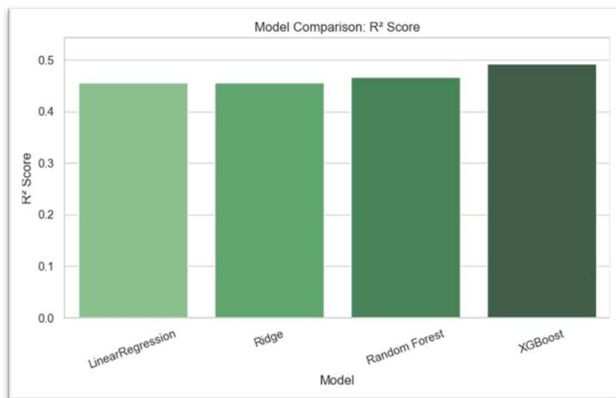
$$\text{dashers_per_order} = \frac{\text{total_onshift_dashers}}{\text{total_outstanding_orders} + 1e^{-5}}$$

c. `Avg_price`: Average item price to reflect complexity. Indicates item complexity by averaging min and max item prices, useful for modeling prep time.

d. `orderPlaced_to_home_location_time`: Represents the estimated time from when the order is placed to when it reaches the customer, based on DoorDash's internal system (i.e., time to place the order plus driving time).

3. Continued outlier removal from Version 2 to ensure robust model training.

Model Performance:



Model	MAE	RMSE	R2
Linear	8.15	10.59	0.4561
Ridge	8.15	10.59	0.4561
Random Forest	8.04	10.49	0.4667
XGBoost	7.84	10.23	0.4926

Observed Changes in Model Performance:

- XGBoost doubled its R^2 from 0.21 to 0.49, explaining nearly half the variance in delivery time.
- All models saw dramatic drops in MAE/RMSE, with XGBoost reducing average ETA error to under 8 minutes.

Key Takeaways from Version 3

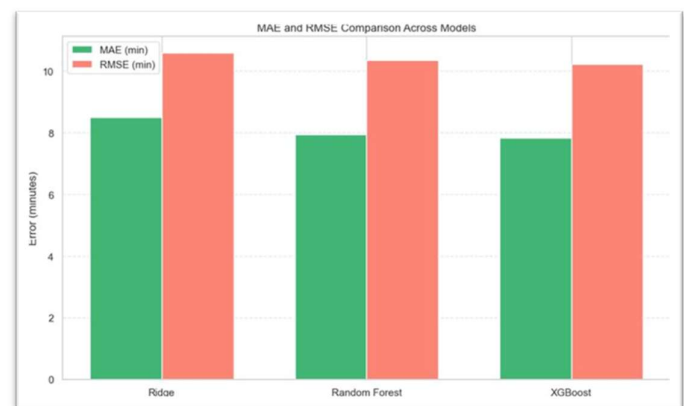
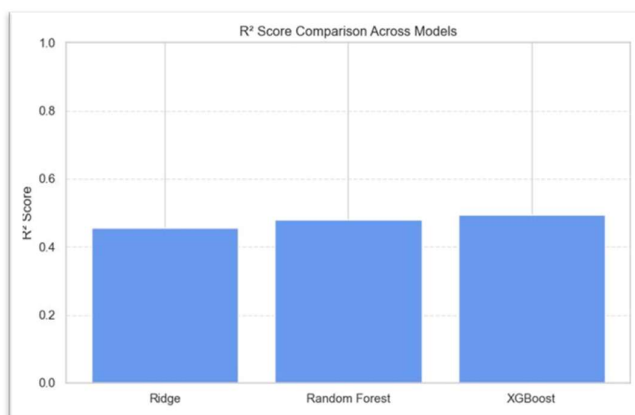
- Version 3 delivered the highest model performance, with XGBoost achieving an R^2 of 0.493, MAE of 7.84, and RMSE of 10.23 — the best results across all models.

- Imputation and feature engineering together improved model performance by retaining the full dataset and enhancing the quality of input features.
- Finally, we performed hyperparameter tuning on all models, including XGBoost, Ridge, and Random Forest, to test for additional performance gains.

Hyperparameter tuning results:

- To further enhance model performance, we applied hyperparameter tuning using cross-validation for Ridge Regression, Random Forest, and XGBoost.
- Linear Regression was excluded from this phase, as it consistently produced the same results as Ridge but without the benefits of regularization. We retained Ridge as the representative linear model.

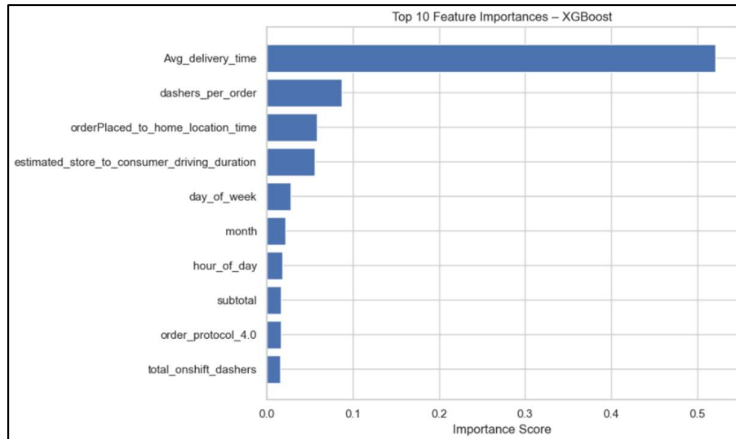
Model	MAE	RMSE	R2	Mean CV R2
Ridge	8.15	10.59	0.4561	0.4634
Random Forest	7.94	10.35	0.4808	0.4865
XGBoost	7.83	10.22	0.4938	0.5015



After tuning:

- XGBoost emerged as the top-performing model, with a test set R^2 of 0.4938 and a mean cross-validation R^2 of 0.5015, showing high accuracy and stability.
- Ridge Regression's results remained largely unchanged, continuing to deliver consistent performance among linear models, though it lacked the flexibility of tree-based models for capturing complex patterns.
- While tuning led to incremental performance gains, it helped fine-tune model configurations and improve reliability.

Feature Importance Analysis – Best Model i.e., XGBoost



After completing hyperparameter tuning and confirming XGBoost as the best-performing model, we analyzed its feature importances to identify the most influential factors driving delivery duration predictions.

XGBoost identified Avg_delivery_time, dashers_per_order, and delivery estimates as key predictors, reflecting store performance and real-time conditions.

Temporal features like day and hour also contributed, showing that a blend of operational, platform, and timing data best predicts delivery duration.

6. Discussion

(a) Key Takeaways from Data Mining:

- From this project, we learned that data pre-processing has a profound impact on model performance. Initial benchmark results on raw data showed low accuracy across all models, especially for tree-based methods like Random Forest, which even returned a negative R^2 . This emphasized the importance of transforming the data before model application.
- We also discovered that not all pre-processing is equally beneficial. In Version 1, removing nulls reduced dataset size and diversity, leading to poor generalization. In contrast, Version 3's use of intelligent imputation (mean, median, mode) allowed us to retain more training data and produced significantly better results.
- Most importantly, feature engineering had the largest influence on predictive performance. Features like Avg_delivery_time and dashers_per_order effectively captured latent variables such as store efficiency and delivery pressure, which were not explicit in the original data. This led to a substantial increase in model accuracy — from $R^2 = 0.21$ to $R^2 = 0.4938$ for XGBoost.
- PCA reduced dimensionality but didn't improve accuracy, and it reduced interpretability—highlighting the need to align preprocessing with model and domain needs.
- Finally, we saw that hyper-parameter tuning helps refine models, but most performance gains came from better data handling. This underlined the idea that “better data beats fancier algorithms” in many real-world machine learning tasks.

(b) Business Implications and Analysis:

The results have several implications for business stakeholders at DoorDash:

- Improved ETA Accuracy: Accurate predictions directly contribute to customer satisfaction and reduce churn. With XGBoost's MAE reduced to under 8 minutes, the platform can offer users more reliable delivery windows.
- Operational Insights: Features like `dashers_per_order` provide real-time indicators of delivery bottlenecks. This insight can be used to dynamically adjust dasher allocation or notify customers proactively about delays.
- Strategic Store Management: `Avg_delivery_time` helps identify slow-performing restaurants, guiding decisions on partnerships or service level agreements (SLAs).
- Data-Driven Decisions: This project shows how historical order data, when processed and modeled properly, can inform everything from customer communication strategies to logistics planning.

(c) Other Recommendations:

- Integrate Live Data Streams: Real-time traffic, dasher GPS location, and restaurant prep time could be integrated into the model to make even more precise ETA predictions. This aligns with the techniques and methodologies currently being used in industry.
- Monitor Model Drift: Over time, customer behaviour and operational dynamics will change. It's important to regularly retrain and validate the model to avoid concept drift.
- Segmented Modelling: Building separate models for different markets, cuisines, or order sizes may further improve accuracy by reducing noise caused by heterogeneity in the data.
- Deploy as Feedback Loop: Incorporating this model into DoorDash's ETA system can enable continual learning by comparing predicted and actual delivery durations.

7. References

- Chen, T. & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. In Proc. of the 22nd ACM SIGKDD Conference (available via arXiv) arxiv.org.
- Scikit-learn community (2023). *Ridge Regression* (`sklearn.linear_model.Ridge`) – API documentation (v1.6.1) scikit-learn.org.
- Scikit-learn community (2023). *PCA: Principal Component Analysis* (`sklearn.decomposition.PCA`) – API documentation (v1.6.1) scikit-learn.org.
- LatentView Analytics (2024). *Food Delivery Apps and Restaurant Churn: Industry Overview, Reasons, and Prevention* (whitepaper) latentview.com.
- Lu, D. & Parekh, P. (2021). *Improving ETA Prediction Accuracy for Long-tail Events*. DoorDash Engineering Blog careersatdoordash.com.
- Official DoorDash Engineering Blog:
 - <https://careersatdoordash.com/blog/doordash-fast-travel-estimates/>
 - <https://careersatdoordash.com/blog/deep-learning-for-smarter-eta-predictions/>